

# Class 6: R functions

Mankeerat Rataul

## Table of contents

|                                                                                           |   |
|-------------------------------------------------------------------------------------------|---|
| Background . . . . .                                                                      | 1 |
| Q1. Write your first R function: <code>add()</code> . . . . .                             | 1 |
| Q2. Write a <code>generate_dna()</code> function . . . . .                                | 2 |
| Q3. Write a <code>generate_protein()</code> function . . . . .                            | 4 |
| Q4. Generate random protein sequences of length 6 to 13 . . . . .                         | 4 |
| Q5. BLASTp search against nr — are your peptides “unique in nature”? . . . . .            | 5 |
| Q6. Connecting your findings to immunology (MHC class II and T-cell activation) . . . . . | 6 |

## Background

All functions in R have at least 3 things:

- a name (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

## Q1. Write your first R function: `add()`

Q1a. Your first version of the function should add two input numbers together. For example, `add(4, 7)` should return 11.

Here we will create a function to add some numbers. Let's call it `add()`

Input arguments can be either “required” or “optional”. The latter will have a default “fall-back” values that will be used if the user doesn't specify it.

```
add <- function(x,y=0) {  
  x + y  
}
```

Can we use our new function:

```
add(37, 45)
```

```
[1] 82
```

Q1b. For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add(c(4,7,10))`.

```
add2 <- function(x,y=0) {  
  sum(x,y)  
}
```

```
add2(c(4,7,10))
```

```
[1] 21
```

Q1c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return 2. Hint: explore the `...` (dots) argument or the base R function `sum()`.

```
add3 <- function(x,y,...) {  
  sum(x,y,...)  
}
```

```
add3(1,2, 3 -4)
```

```
[1] 2
```

## Q2. Write a `generate_dna()` function

A useful function here is the “base R” `sample()` function:

```
sample(1:5, size=6, replace=TRUE)
```

```
[1] 2 1 3 4 3 5
```

Q2a. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”.

```
#defines the nucleotides to be used later
nucleotides <- c("A", "C", "G", "T")

#function to generate random DNA sequence by using sample, and using replace=TRUE to allow i
generate_dna <- function(DNA_len) {
  sample(nucleotides, size=DNA_len, replace=TRUE)
}
# calls upon function and generates e.g. "T" "G" "A" "G" "A" "G"
generate_dna(6)
```

```
[1] "C" "G" "T" "C" "G" "G"
```

Q2b. Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a single character string (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”.

```
# Makes a new function to paste the nucleotides together once sampled. Makes the default len
generate_dna2 <- function(DNA_len=6) {
  paste(sample(nucleotides, size=DNA_len, replace=TRUE), collapse="")
}
#Calls function, results in e.g. "GGTGGC"
generate_dna2(6)
```

```
[1] "GACCAA"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example: >len9  
CGAAGGCTG

```
#creates new function for the FASTA creation
generate_dna3 <- function(DNA_len) {

  # Returns the >len# of the sequence followed by a line break and then the DNA seq
  cat(">", "len", DNA_len, "\n", sample(nucleotides, size=DNA_len, replace=TRUE), collapse="")
}
#calls the function finally, e.g. >len6 [enter] ACTATA
generate_dna3(6)
```

```
>len6
CCATTG
```

### Q3. Write a generate\_protein() function

```
# Defines the amino acids for future use
amino_acids <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T", "W", "V", "Y")

#Function to paste the amino acids together into protein, default length as 10
generate_protein <- function(peptide_len=10) {
  paste(sample(amino_acids, size=peptide_len, replace=TRUE), collapse="")
}
#Calls function, results in e.g. "PNSTWLKCDK"
generate_protein(10)
```

```
[1] "VLPPSRDWIF"
```

### Q4. Generate random protein sequences of length 6 to 13

```
# Create new function
generate_protein2 <- function(peptide_len=10){
  # Concatenates the >len, length of the sequence, adds a line break, then adds the sequence
  cat(">len", peptide_len, "\n", sample(amino_acids, size=peptide_len, replace=TRUE), "\n", collapse="")
}

# Runs generate_protein() for sequence lengths 6-13
for(i in 6:13) {
  generate_protein2(i)
}
```

```

>len6
AMHLQK
>len7
MWGHEVY
>len8
GNWVTQLT
>len9
TAMWPWIPR
>len10
LNPQYTRGFP
>len11
QWTWIEYCLCV
>len12
DWGYDKLVMLFG
>len13
AKQYWGLLCKMQT

```

#### Q5. BLASTp search against nr — are your peptides “unique in nature”?

| Length (aa) | Best hit % identity | Best hit % coverage | Unique? (Y/N) |
|-------------|---------------------|---------------------|---------------|
| 6           | 100%                | 100%                | N             |
| 7           | 100%                | 100%                | N             |
| 8           | 87.50%              | 100%                | Y             |
| 9           | 100%                | 89%                 | Y             |
| 10          | 77.78%              | 90%                 | Y             |
| 11          | 90%                 | 91%                 | Y             |
| 12          | 100%                | 75%                 | Y             |
| 13          | 71.43%              | 92%                 | Y             |

Q5a. At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)?

They start to look unique at a peptide length of 8, where the protein I got had only 87.50% coverage.

Q5b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space ( $20^n$  for a peptide of length  $n$ ) and to the size of the known protein universe

The main reason is because shorter sequences have a higher probability of being common due to the low level of amino acids, so there is likely coincidental sequences that are identical, whereas as a peptide gets longer, it becomes more unlikely for them to be exactly the same.

**Q6. Connecting your findings to immunology (MHC class II and T-cell activation)**

In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

The minimum length may be 8 amino acids long, as 7 and 6 are too short and will have a lot of coincidental binding and thus T cell activation, causing a whole host of autoimmune problems. Having the immune system present short peptides is harmful for this very reason, since self proteins can also be in common with those of pathogens at that low of amino acid length and can cause wrongful activation. Longer peptides have a lower chance of being coincidentally the same, so they're much more reliable for binding and activating T cell receptors in the right situations.